

BLUEBIRD

Hotel Management System

Complete report presentation: requirements, specification, design, implementation, testing, maintenance, retirement, screenshots and appendices.

PHP / MySQL • SDLC • UML • Testing

to heaven on earth

BLUEBIRD

Hotel Management System

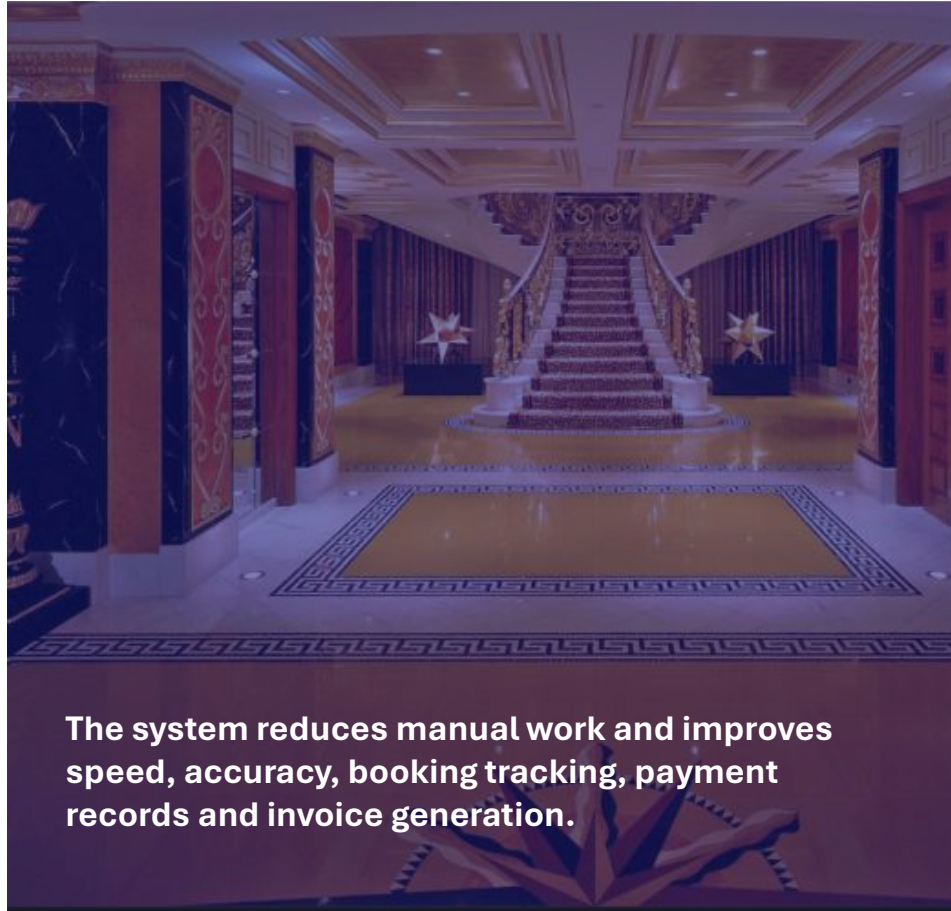
PHP / MySQL • SDLC • UML • Testing

	Name	ID
1	Hossam Ahmed Adel	2305037
2	Antony Medhat Makram	2305065
3	Mohamed Ibrahim Hussein	2305026
4	Youssef Ahmed Wahid	2305105
5	Mayer Adel Kher	2305083
6	Rawan Sherif Ibrahim	2305144

to heaven on earth

Executive Summary

BLUEBIRD centralizes customer reservations and hotel administration in one web application.



Users

Register, log in, view hotel rooms and facilities, and submit reservations.

Admins / Staff

Manage bookings, confirm reservations, generate payments, print invoices, manage rooms and staff.

System scope

Standalone PHP/MySQL web app using the bluebirdhotel database and XAMPP local environment.

Documentation

Full SDLC report with requirements, SRS, UML diagrams, testing, maintenance, retirement and appendices.

Presentation Roadmap

The deck follows the same structure as the project report and ends with the original full-page report appendix.

01 Introduction, Description, Objectives, Stakeholders

02 Process phases and Requirements Analysis

03 SRS: environment, inputs, use cases and traceability

04 Design: architecture, modules, database and UML diagrams

05 Implementation: tools, source code and coding notes

06 Testing, Maintenance, Retirement and Appendices

Introduction: Problem and Solution

Manual hotel administration can cause duplicate records, delayed confirmation and inaccurate payment calculation.

Manual administration problems

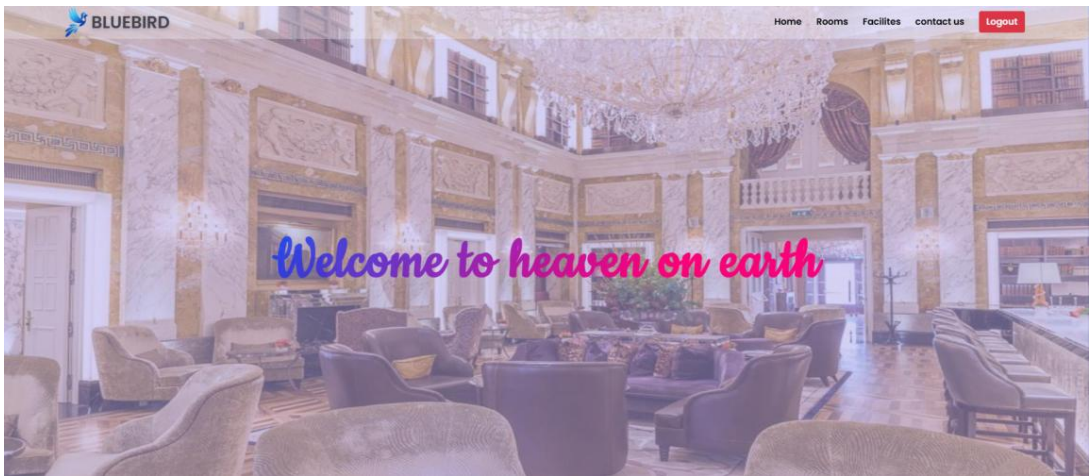
Duplicate records, delayed confirmation, inaccurate payment calculation, and difficulty tracking room booking status.

Centralized web application

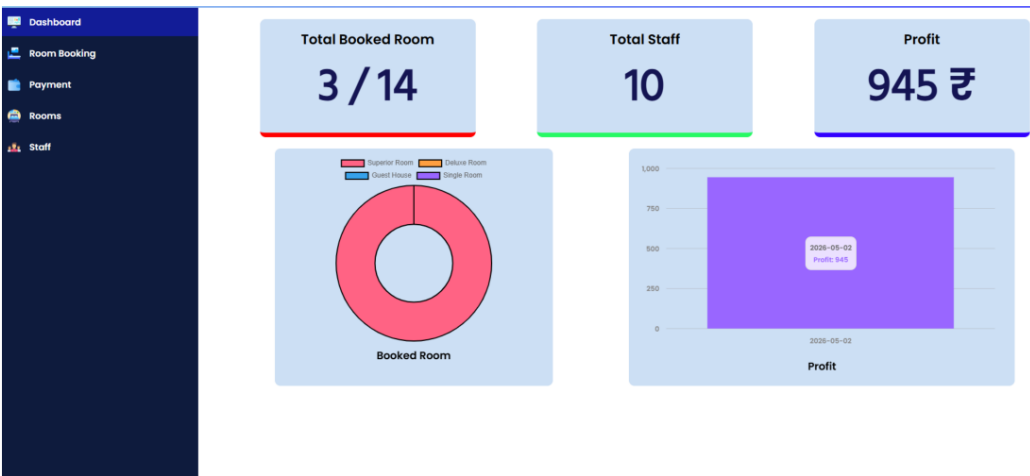
A browser-based system that organizes reservations, customer data, room records, payments, invoices and staff data.

Administration panel

Admin/staff monitor dashboards, confirm reservations, calculate payments, print invoices and manage rooms/staff.



User homepage



Admin dashboard

Project Objectives

The system objectives focus on booking, administration, automation, invoices, statistics and documentation.

Customer access

Allow customers to register, log in and submit hotel room reservations.

Admin operations

Allow admin/staff to manage reservations, rooms, staff and payments.

Automation

Automate payment calculation after booking confirmation.

Invoices

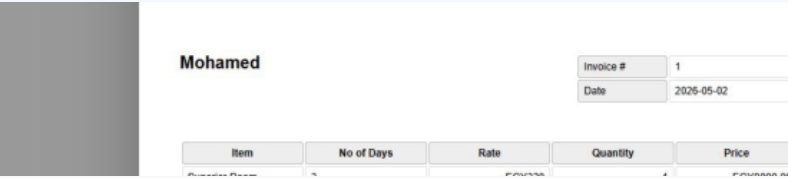
Generate printable invoices for confirmed bookings.

Dashboard

Summarize reservations, rooms, staff and profit through dashboard statistics.

Course documentation

Document the complete development process using SDLC phases and UML diagrams.



The screenshot shows a printable invoice for a customer named Mohamed. It includes fields for Invoice # (1) and Date (2026-05-02). Below these is a table with columns: Item, No of Days, Rate, Quantity, and Price. The table contains one row of data.

Mohamed				
		Invoice #	1	
		Date	2026-05-02	
Item	No of Days	Rate	Quantity	Price
Room #101	5	20000	1	100000

Printable invoice output from the report screenshots

Stakeholders and User Classes

The report identifies four stakeholder groups and their main interests.

Guest / Registered User

Hotel customer who uses the website to view facilities and reserve a room.

Needs easy booking, a clear interface and accurate reservation data.

Admin / Staff

Hotel employee who manages bookings, rooms, staff, payments and invoices.

Needs efficient management and accurate operational records.

Hotel Management

Decision makers who monitor business status through reports and dashboard totals.

Need reliable records, profit visibility and control.

Development Team

Designers, implementers, testers and documentation owners.

Need clear requirements, maintainable code and correct diagrams.

Process Phases

The SDLC process maps theoretical phases to project application.

Phase	Theoretical meaning	Project application
Requirements Analysis	Understand customer/user needs and capture data, functionality and quality attributes.	Identified user/admin needs, scope, stored data, functions and quality requirements.
Specification	Describe exactly what the product will do, with inputs, outputs and constraints.	Prepared SRS-style details, use-case specifications, traceability and sign-off placeholders.
Design	Describe structure with high-level/detail design and UML models.	Prepared architecture, module design, database design and UML diagrams.
Implementation	Implement the detailed design in code and document/test cases.	Used PHP, MySQL, HTML, CSS, Bootstrap, JavaScript and XAMPP modules.
Integration	Integrate modules and perform interface, product and acceptance tests.	Integrated login, booking, confirmation, payment, room, staff and invoice modules.
Maintenance	Apply adaptive, perfective, corrective and preventive changes.	Outlined future fixes, security updates, backup and regression testing.
Retirement	Replace/rewrite when unmaintainable or incompatible.	Defined retirement triggers for the Hotel Management System.

Requirements Elicitation

The analysis phase used prototyping, observation, interviews, discussion and document analysis.

Technique	How it was used	Result
Prototyping	Login, booking, dashboard, payment and invoice pages were treated as a rapid prototype.	Improved understanding of required system behavior before final documentation.
Observation	Common hotel workflow: customer requests reservation, staff confirms, payment is calculated and invoice is printed.	Produced the main workflow used in activity and sequence diagrams.
Interviews	Assumed questions for staff/admin and customers about reservation and payment data.	Identified fields: name, email, phone, room type, bed, meal, check-in/out and status.
Workshop / Team Discussion	Team members discussed scope and selected manageable modules for a course project.	Defined included and excluded features.
Document Analysis	Existing PHP files and SQL database were reviewed.	Mapped implemented modules to requirements and test cases.

System Scope

The system boundary is limited to local hotel booking data and administration records.

In scope

User registration and login, admin/staff login, room browsing, reservation submission, booking list, booking confirmation, payment generation, invoice printing, room management, staff management, dashboard statistics and export of booking data.

Out of scope

Online payment gateway, email/SMS notifications, multi-hotel branches, real-time payment integration, mobile application and advanced role-based permissions.

System boundary

The application manages hotel booking data and administration records inside the bluebirdhotel database. It does not communicate with external banking or messaging services.

Data Requirements

The database stores accounts, room inventory, booking details, payment totals and staff records.

Data entity	Stored data
User account	UserID, username, email, password.
Admin/staff login	Employee ID, employee email, employee password.
Room	Room ID, room type, bedding type.
Booking	Booking ID, customer name, email, country, phone, room type, bed, meal, number of rooms, check-in, check-out, number of days, status.
Payment	Payment ID, customer details, room type, bed, number of rooms, dates, number of days, room total, bed total, meal total, final total.
Staff	Staff ID, staff name, work type.

Functional Requirements: Access and Booking Review

Must-have requirements FR-01 to FR-08 define user/admin access and booking confirmation.

ID	Requirement	Actor	Priority
FR-01	Allow a new user to sign up using username, email, password and confirm password.	User	Must
FR-02	Allow registered users to log in using email and password.	User	Must
FR-03	Allow admin/staff to log in using employee email and password.	Admin/Staff	Must
FR-04	Display hotel rooms and facilities to the user.	User	Must
FR-05	Allow the user to submit room reservation details.	User	Must
FR-06	Store each new booking with status NotConfirm until reviewed by admin.	System	Must
FR-07	Allow admin to view all room bookings.	Admin/Staff	Must
FR-08	Allow admin to confirm a booking.	Admin/Staff	Must

Functional Requirements: Payments, Invoices and Management

Requirements FR-09 to FR-15 cover automation, administration and optional export.

ID	Requirement	Actor	Priority
FR-09	Calculate room, bed, meal and final totals after booking confirmation.	System	Must
FR-10	Store the payment record in the database after booking confirmation.	System	Must
FR-11	Allow admin to print invoice details for a payment record.	Admin/Staff	Should
FR-12	Allow admin to add and delete room records.	Admin/Staff	Should
FR-13	Allow admin to add and delete staff records.	Admin/Staff	Should
FR-14	Display dashboard statistics for bookings, rooms, staff and profit.	Admin/Staff	Should
FR-15	Allow admin to export booking data.	Admin/Staff	Could

Non-functional Requirements

Quality requirements are measurable and connected to verification methods.

ID / Quality	Measurable requirement	Verification
NFR-01 Performance	Pages load within 3 seconds on local XAMPP for normal course-project data volume.	Stopwatch or browser dev tools.
NFR-02 Security	Admin pages accessible only after successful admin/staff login session.	Try direct access without session.
NFR-03 Usability	A new user can complete booking form without special training.	Usability test with at least one user.
NFR-04 Reliability	A valid booking is stored correctly in roombook without losing required fields.	Compare form data with database record.
NFR-05 Availability	System available while Apache and MySQL services are running.	Run XAMPP and access website.
NFR-06 Maintainability	Pages separated into user, admin, CSS, JavaScript and database files.	Review project folder structure.
NFR-07 Scalability	Database allows adding more rooms, bookings, staff and payments without main page changes.	Insert new records and verify display.

SMART Review and MoSCoW Prioritization

Requirements use clear verbs, measurable checks and priority groups.

SMART	Application
Specific	Requirements use verbs such as register, log in, submit booking, confirm booking, calculate payment, print invoice, add room and delete staff.
Measurable	NFRs include measurable targets such as load time and data storage verification.
Achievable	Scope is limited to a PHP/MySQL web app suitable for a course project.
Relevant	Each requirement supports a real hotel booking or administration activity.
Time-based	Must-Have features are prioritized before optional improvements.

Priority	Hotel Management System requirements
Must-Have	User login, admin login, booking submission/storage/confirmation, payment calculation, database connection.
Should-Have	Invoice printing, room management, staff management, dashboard statistics.
Could-Have	Export booking data, improved search/filtering, richer reports.
Would-Have / Won't-Have now	Online payment gateway, email/SMS confirmation, mobile app, multi-branch support.

Ambiguity, Contradiction and Incompleteness Review

The report converts unclear ideas into testable and maintainable specifications.

Possible issue	Risk	Resolution
“Manage booking” is ambiguous.	Could mean create, view, edit, confirm, delete or search.	Separated view, confirm, delete and edit booking into distinct functions.
“System should be fast” is vague.	Cannot be tested objectively.	NFR-01 defines 3 seconds in local XAMPP for normal data volume.
Room availability can be incomplete.	Booking may be accepted even if room count is insufficient.	Availability checking is partially reflected; stronger validation is future improvement.
Password security is incomplete.	Plain text passwords are a security weakness.	Future improvement: use password_hash() and password_verify().
Admin access control may be inconsistent.	Direct access may bypass navigation if session checks are missing.	Security testing and future role-based access control are included.

Specification Phase: Product Perspective and Environment

The system is a standalone web-based hotel administration app running on PHP/MySQL.

Product perspective

Users access the website through a browser. Admin/staff access management functions through an admin dashboard. MySQL stores all persistent data.

Component	Environment
Web server	Apache server through XAMPP.
Database server	MySQL/MariaDB, database name: bluebirdhotel.
Client	Modern browser such as Chrome, Edge, Firefox or equivalent.
Programming language	PHP backend; HTML, CSS, Bootstrap and JavaScript interface.

Developer 2

Antony Medhat	Approved	Antony Medhat / 28-4-26
---------------	----------	-------------------------

High-level architecture from the report

Inputs, Processes, Outputs and Constraints

The SRS specifies what data enters the system, how it is processed and what outputs appear.

Area	Specification
Inputs	Login credentials, signup details, customer contact data, room type, bed type, meal, number of rooms, check-in/out dates, room/staff entries and admin actions.
Processes	Credential validation, booking insertion, room count checking, status update, number-of-days calculation, payment total calculation, dashboard statistics and invoice display.
Outputs	Homepage, booking messages, booking table, dashboard cards/charts, payment table, printable invoice, room list, staff list and exported booking data.
Constraints	Local PHP/MySQL deployment; no external payment gateway; admin actions depend on session/database connection; final report requires UML diagrams.

Use Case Specifications

Main scenarios describe actor, precondition, success path and exceptions.

Use Case	Actor	Precondition	Main success scenario	Alternative / Exception
Register User	Guest	Guest is not logged in.	Enter username, email, password and confirm password; system validates and creates account.	Email exists or passwords mismatch: error.
Login User	Registered User	User has account.	Enter email/password; system validates; homepage opens.	Invalid credentials: login error.
Book Room	User	User is logged in.	Select room, bed, meal, rooms, dates; system stores NotConfirm booking.	Missing fields: correction required.
Confirm Booking	Admin/Staff	Admin logged in and booking exists.	Admin confirms; system updates status and creates payment record.	Already confirmed: duplicate prevented.
Print Invoice	Admin/Staff	Payment record exists.	Admin opens invoice print page; invoice and print option display.	Missing payment: cannot generate invoice.
Manage Rooms	Admin/Staff	Admin logged in.	Admin adds or deletes room record.	Invalid room data rejected.
Manage Staff	Admin/Staff	Admin logged in.	Admin adds or deletes staff record.	Invalid staff data rejected.

Main Success Scenario

The A/S style flow connects login, booking, admin confirmation, payment and invoice.

- 1

A: User enters email and password.
- 2

S: System validates credentials against signup table.
- 3

S: System opens homepage for valid user.
- 4

A: User enters room booking details and submits.
- 5

S: System validates required booking fields.
- 6

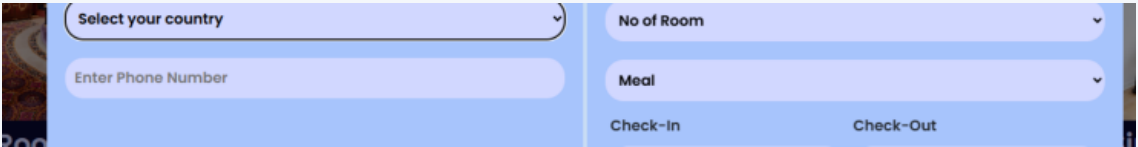
S: System stores booking in roombook with NotConfirm.
- 7

A: Admin opens booking page and clicks Confirm.
- 8

S: System updates status to Confirm.
- 9

S: System calculates room, bed, meal and final totals.
- 10

S: System stores payment and allows invoice printing.



The screenshot shows a portion of a web form for hotel booking. On the left, there is a dropdown menu labeled 'Select your country' and a text input field labeled 'Enter Phone Number'. On the right, there are two more dropdown menus labeled 'No of Room' and 'Meal'. Below these, there are labels for 'Check-In' and 'Check-Out' next to their respective input fields.

Requirements Traceability Matrix

Traceability links requirements to modules, implementation files and test cases.

Req. ID	Design element / Module	Implementation file(s)	Test case
FR-01	Authentication module	index.php, signup table	TC-01
FR-02	Authentication module	index.php, signup table	TC-02
FR-03	Admin authentication module	index.php, emp_login table	TC-03
FR-05/FR-06	Booking module	home.php, roombook table	TC-04
FR-07/FR-08	Booking management module	admin/roombook.php, admin/roomconfirm.php	TC-05, TC-06
FR-09/FR-10	Payment module	admin/roomconfirm.php, admin/payment.php, payment table	TC-07
FR-11	Invoice module	admin/invoiceprint.php	TC-08
FR-12	Room management module	admin/room.php, room table	TC-09, TC-10
FR-13	Staff management module	admin/staff.php, staff table	TC-11, TC-12
FR-14	Dashboard module	admin/dashboard.php	TC-13

Requirements Sign-off and High-Level Architecture

Sign-off records approvals and the architecture explains the flow between browser, PHP modules and MySQL.

Role	Name	Approval status	Signature / Date
Client / Instructor	NULL	Pending	NULL
Team Leader	Hossam Ahmed	Approved	Hossam Ahmed / 01-05-26
Developer 1	Mohamed Ibrahim	Approved	Mohamed Ibrahim / 28-4-26
Developer 2	Antony Medhat	Approved	Antony Medhat / 28-4-26

Developer 2	Antony Medhat	Approved	Antony Medhat / 28-4-26
-------------	---------------	----------	-------------------------

Architecture summary

Presentation tier sends HTTP requests/responses to the application tier. PHP pages and modules perform authentication, booking, payment, room and staff logic. The data tier stores SQL queries, records and reports in the bluebirdhotel database.

Module Decomposition

Each module has a responsibility, main input and main output.

Module	Responsibility	Main input	Main output
Authentication	User signup, user login, admin/staff login.	Email, password, username.	Authenticated session or error.
Booking	Capture reservation details and store booking.	Customer data, room, bed, meal, dates.	Booking record with NotConfirm.
Booking Management	Admin views, edits, confirms or deletes bookings.	Booking ID and admin action.	Updated status or deleted record.
Payment	Calculate and display payment totals.	Confirmed booking data.	Payment record and final total.
Invoice	Display printable invoice.	Payment ID.	Invoice page.
Room Management	Add/delete room records.	Room type and bedding.	Updated room list.
Staff Management	Add/delete staff records.	Staff name and work.	Updated staff list.
Dashboard	Summarize business data.	Booking, payment, room, staff records.	Dashboard cards and charts.

Database Design

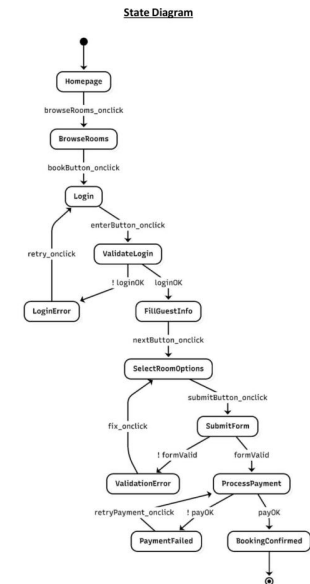
The database uses separate tables for users, staff/admin login, rooms, bookings, payments and staff.

Table	Purpose	Main columns
signup	Stores user accounts.	UserID, Username, Email, Password
emp_login	Stores admin/staff login accounts.	empid, Emp_Email, Emp_Password
room	Stores available room type and bedding combinations.	id, type, bedding
roombook	Stores reservation details and booking status.	id, Name, Email, Country, Phone, RoomType, Bed, Meal, NoofRoom, cin, cout, nodays, stat
payment	Stores calculated payment and invoice-related totals.	id, Name, Email, RoomType, Bed, NoofRoom, cin, cout, noofdays, roomtotal, bedtotal, meal, mealtotal, finaltotal
staff	Stores staff records.	id, name, work

Design Decisions

Key decisions support deployment simplicity, maintainability and data organization.

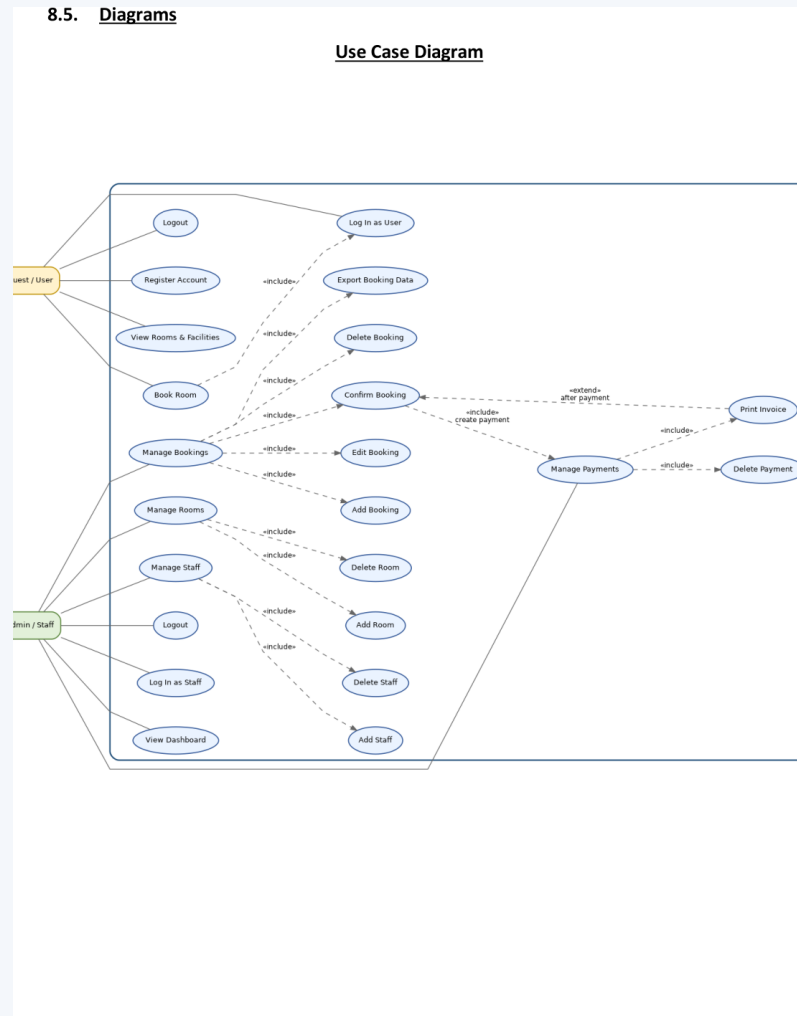
- PHP/MySQL architecture selected because it is simple to deploy using XAMPP and suitable for the course-project scope.
- Separate admin pages for dashboard, booking, payment, room and staff modules improve maintainability.
- Bookings start with status NotConfirm and move to Confirm only after admin review.
- Payment is generated after booking confirmation because it depends on room type, bed, meal, rooms and days.
- Separate database tables keep users, staff/admin login, rooms, bookings, payments and staff organized.



State transition logic

UML Diagrams: Use Case Diagram

The use case diagram shows guest/user actions and admin/staff management actions.



Guest/User side

Login, register account, view rooms and facilities, book room and logout.

Admin/Staff side

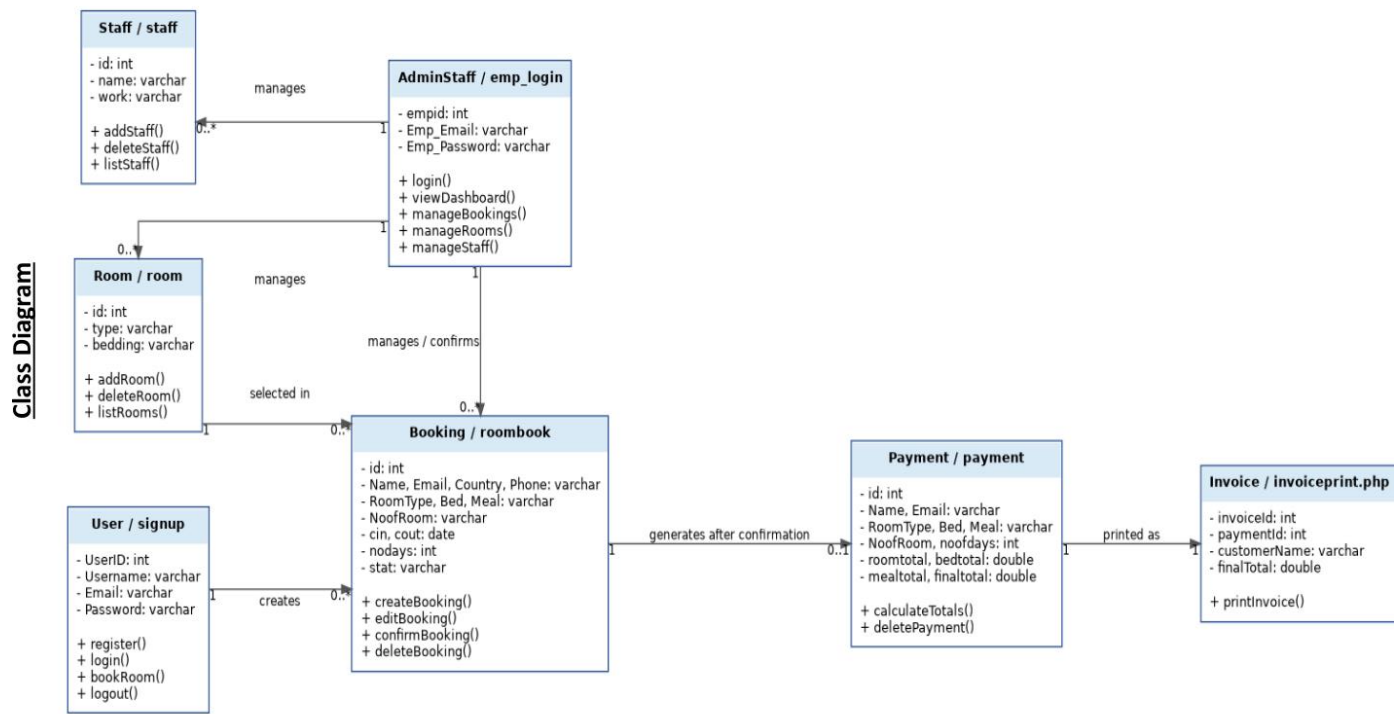
Manage bookings, rooms, staff, payments, invoices, dashboard and logout.

Included actions

Export booking data, confirm/delete/edit booking, add/delete rooms and staff, print/delete payment.

UML Diagrams: Class Diagram

The class model shows the main entities and relationships behind reservations, payments and invoices.



Key classes

Guest/User, Admin/Employee, Room, Booking/Reservation, Payment and Invoice.

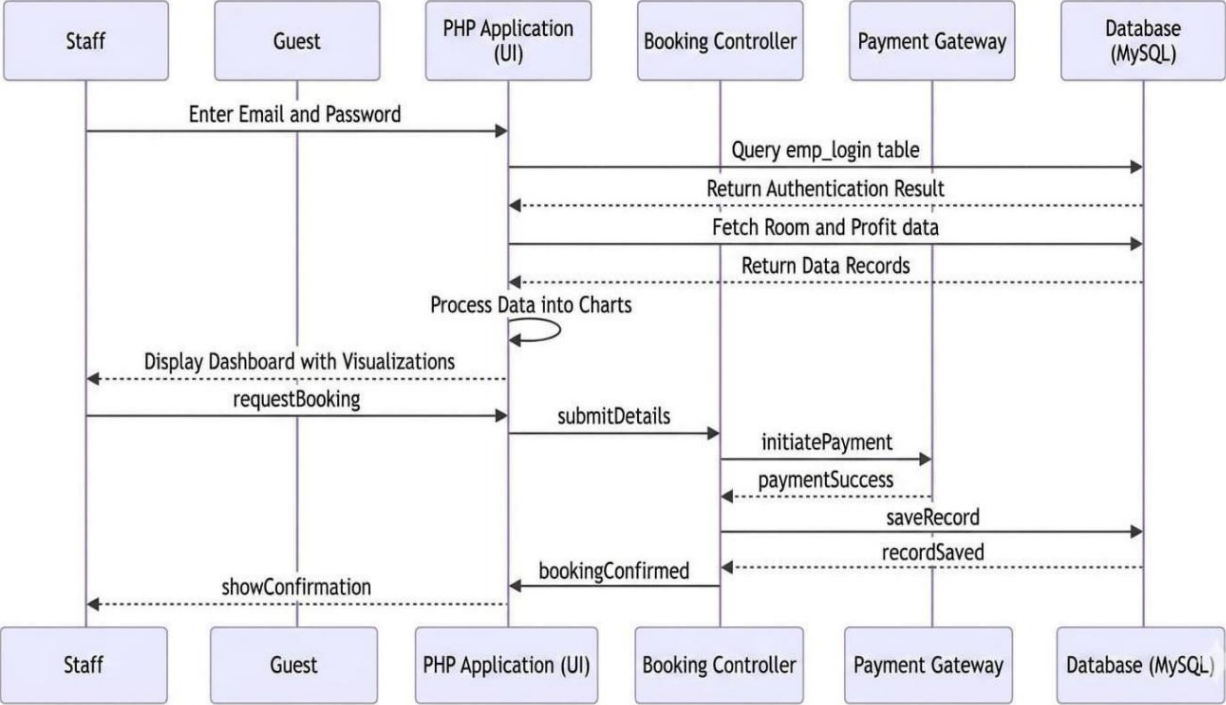
Core relationship

A guest makes bookings, room availability is managed, a confirmed booking generates payment and a payment produces an invoice.

UML Diagrams: Sequence Diagram

The sequence diagram follows the interaction between staff/guest, PHP controllers and the MySQL database.

Sequence Diagram



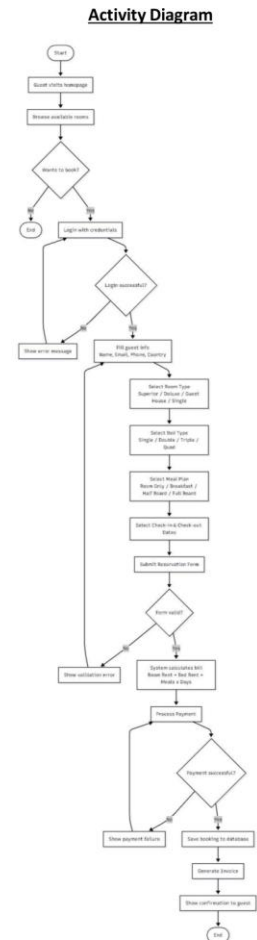
Primary flow

Guest enters credentials, gets authenticated, enters booking details, submits data, receives confirmation and booking record is saved.

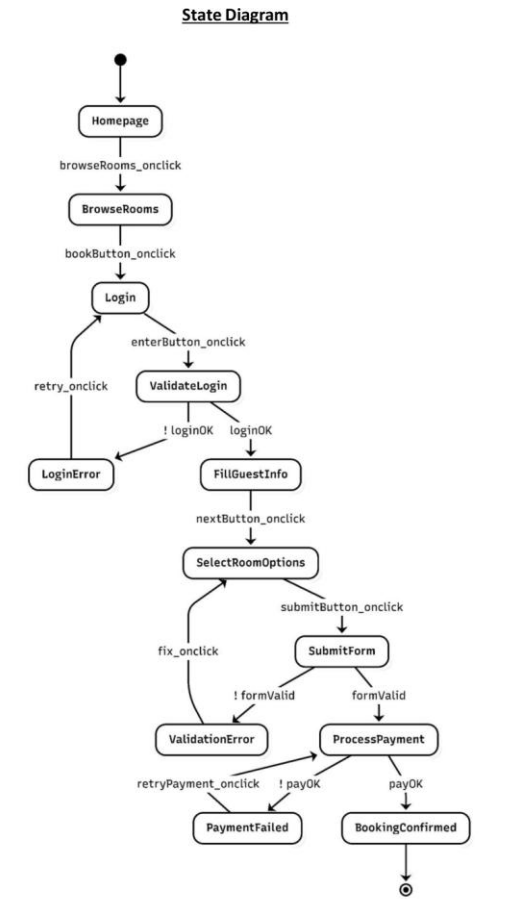
System components

PHP application/UI, Booking Controller, Payment Gateway placeholder and MySQL database participate in the flow.

The activity diagram covers the process flow, while the state diagram models page and booking states.



Activity Diagram: reservation and confirmation process

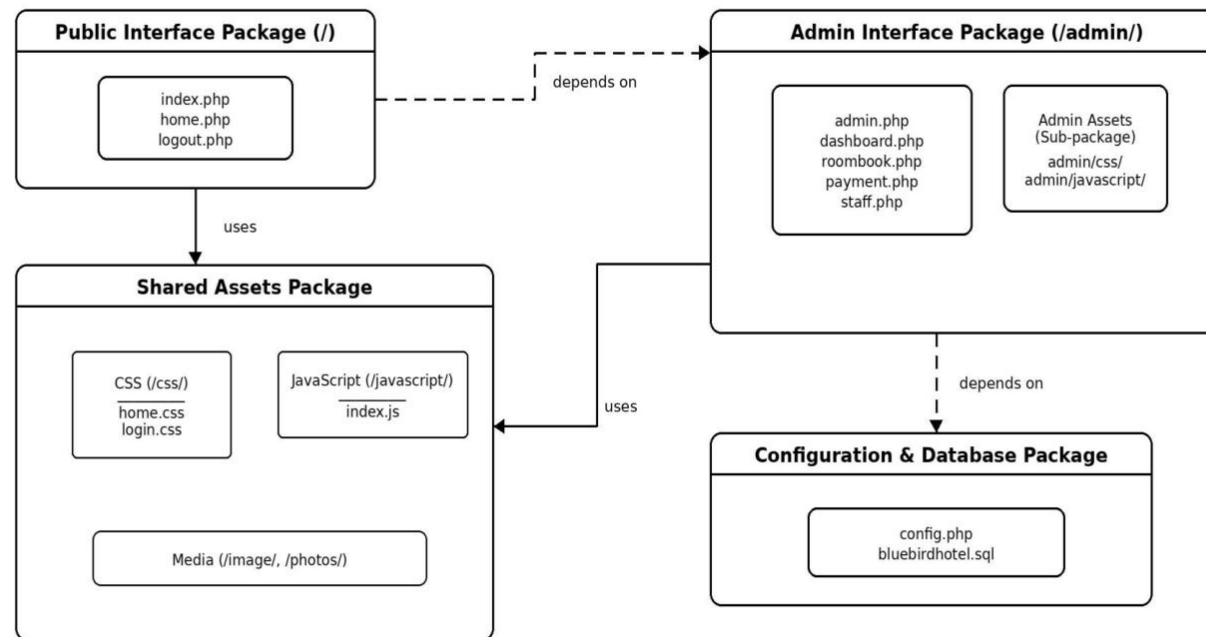


State Diagram: homepage → room browsing → login → booking → payment result

UML Diagrams: Package, Component and Collaboration

Additional UML views show package organization, components and object collaboration.

Package Diagram

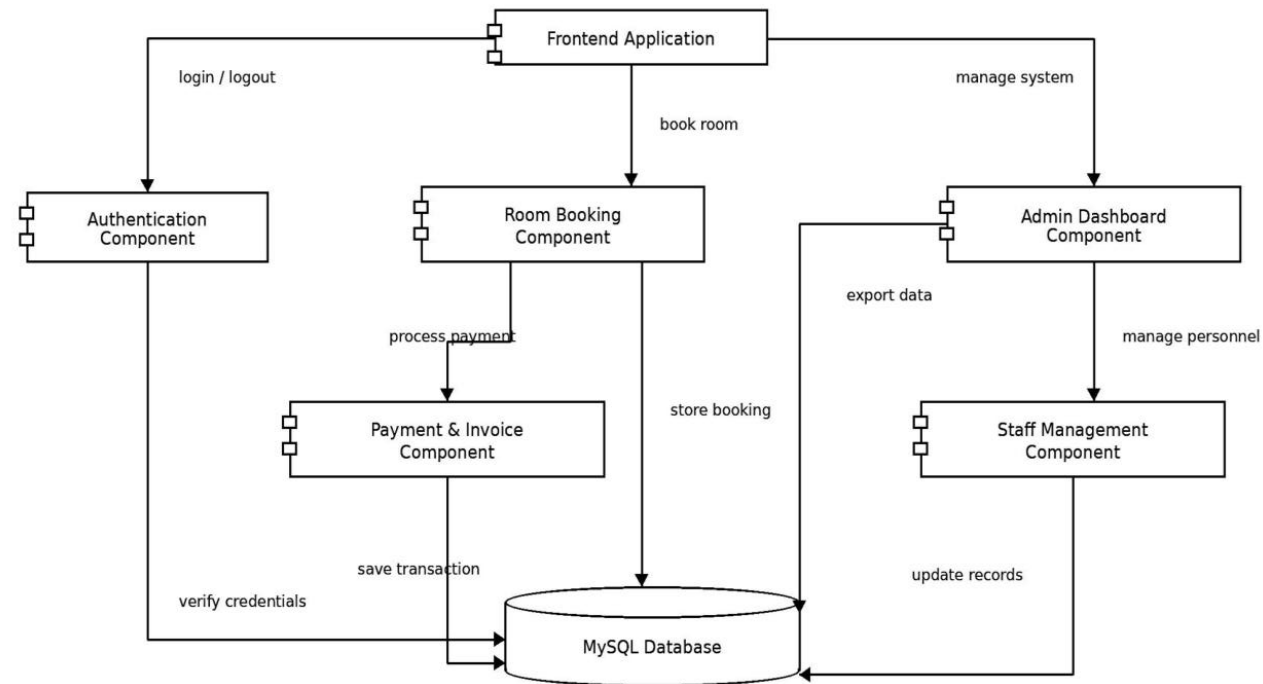


Package Diagram

UML Diagrams: Package, Component and Collaboration

Additional UML views show package organization, components and object collaboration.

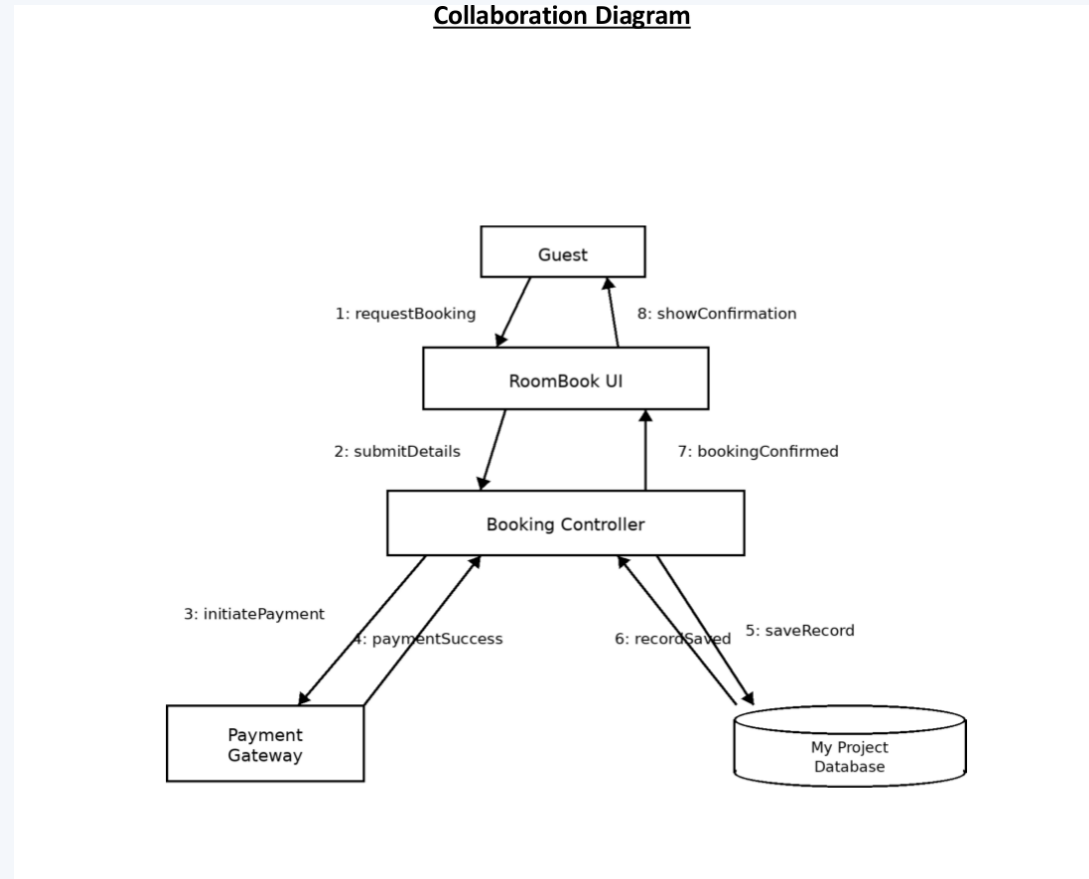
Component Diagram



Component Diagram

UML Diagrams: Package, Component and Collaboration

Additional UML views show package organization, components and object collaboration.



Collaboration Diagram

Implementation: Technologies and Tools

The implementation uses a standard PHP/MySQL course-project stack.

Technology / Tool	Usage in the project
PHP	Backend logic, session handling, database queries and form processing.
MySQL / MariaDB	Persistent storage through the bluebirdhotel database.
HTML	Structure of user and admin pages.
CSS	Custom styling for login and home pages.
Bootstrap	Responsive layout and interface components.
JavaScript	Client-side page interaction and table search.
XAMPP	Local Apache and MySQL development environment.
phpMyAdmin	Database creation, import and inspection.
Draw.io / StarUML / Graphviz equivalent	Computerized UML diagrams.

Source Code Organization

The report maps project files to their responsibilities.

Path / File	Purpose
index.php	User login, user signup and admin/staff login.
home.php	User homepage and booking form.
config.php	Database connection settings.
admin/admin.php	Admin layout with dashboard frame links.
admin/dashboard.php	Dashboard statistics and charts.
admin/roombook.php	Booking management page.
admin/roomconfirm.php	Booking confirmation and payment insertion logic.
admin/payment.php	Payment table and search.
admin/invoiceprint.php	Invoice printing page.
admin/room.php	Room add/delete management.
admin/staff.php	Staff add/delete management.
bluebirdhotel.sql	Database schema and sample data.

Development Steps and Coding Practice Notes

The report documents setup, implementation, testing and improvement recommendations.

- Set up XAMPP and create bluebirdhotel database.
- Import bluebirdhotel.sql using phpMyAdmin.
- Configure database connection in config.php.
- Implement/test user signup and login.
- Implement homepage and booking submission.
- Implement admin dashboard and booking management.
- Implement booking confirmation, payment calculation and insertion.
- Implement invoice, room, staff and export functions.
- Perform syntax checks, manual tests, integration testing and documentation.

Practice	Current status / recommendation
Separation of concerns	Project separates user pages, admin pages, stylesheets, JavaScript, images and database script.
Database connection reuse	config.php centralizes the MySQL connection.
Session handling	Sessions distinguish logged-in users/admins.
Input validation	Basic required-field checks exist; add deeper validation for dates, phone and availability.
Security	Convert remaining direct SQL queries to prepared statements.
Password storage	Use password_hash() and password_verify() in future.

Testing Strategy

Testing covers informal checks, units, integration, system flow, acceptance and regression.

Testing type	Purpose	Example in this project
Desk checking / programmer testing	Informal checking while writing code.	Review PHP conditions, SQL queries and form field names.
Unit testing	Check individual pages/modules.	Test login, signup, add room and add staff separately.
Integration testing	Check module interfaces.	Submit booking then verify it appears in admin booking table.
System testing	Check complete system behavior.	Run full flow from signup to invoice printing.
Acceptance testing	Decide whether delivered system satisfies project objective.	Final team/instructor discussion and demonstration.
Regression testing	Retest old functions after changes.	After payment logic changes, retest login, booking, confirmation and invoice.

Test Cases

All listed test cases in the report are marked Verified.

Test ID	Scenario	Expected result	Status
TC-01	User signup	Account is created and user can log in.	Verified
TC-02	User login	User homepage opens.	Verified
TC-03	Admin login	Admin panel opens.	Verified
TC-04	Submit booking	Booking saved in roombook with NotConfirm status.	Verified
TC-05	View bookings	All bookings appear in table.	Verified
TC-06	Confirm booking	Booking status becomes Confirm.	Verified
TC-07	Generate payment	Payment record created with final total.	Verified
TC-08	Print invoice	Invoice displays customer and total payment details.	Verified
TC-09	Add room	Room record appears in room list.	Verified
TC-10	Delete room	Room record is removed.	Verified
TC-11	Add staff	Staff record appears in staff list.	Verified
TC-12	Delete staff	Staff record is removed.	Verified
TC-13	Dashboard statistics	Counts and totals are displayed.	Verified
TC-14	Invalid login	System shows invalid credentials / access is not granted.	Verified

Maintenance Phase

Maintenance planning includes corrective, adaptive, perfective and preventive work.

Maintenance type	Meaning	Project application
Corrective	Diagnose and fix discovered errors.	Fix wrong table heading, failed redirect, duplicate payment generation or wrong total calculation.
Adaptive	Modify system for environment changes.	Update code for newer PHP/MySQL version or different hosting server.
Perfective	Add/improve functionality based on user needs.	Add online payment, email confirmation, room availability calendar or monthly reports.
Preventive	Improve maintainability/reliability before failure.	Use prepared statements everywhere, hash passwords, refactor repeated code and add backups.

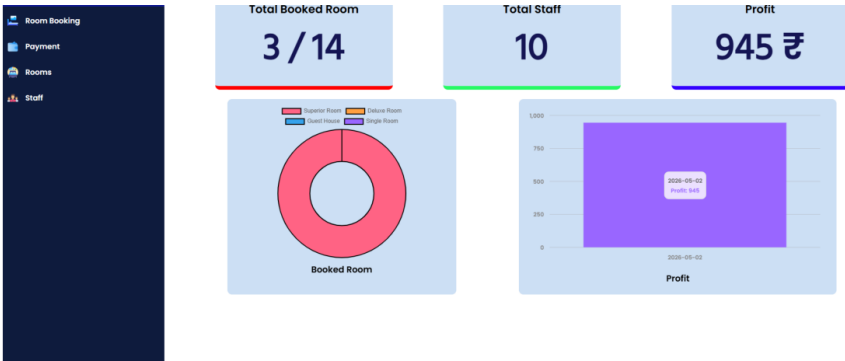
Future Improvements and Regression Testing

The report lists security, validation, workflow, reporting and deployment improvements.

- Use password hashing for all user/admin passwords.
- Convert all direct SQL queries to prepared statements.
- Add stricter validation for dates, phone, room count and availability.
- Add customer booking history and cancellation flow.
- Add email/SMS confirmation after reservation/payment.
- Add online payment gateway integration.
- Add role-based access control for staff responsibilities.
- Add monthly/yearly financial reports and downloadable invoices.
- Improve mobile responsiveness of admin pages.
- Deploy on live HTTPS hosting.

Regression suite

After each maintenance change, rerun user login, booking submission, admin login, booking confirmation, payment generation, invoice printing, add/delete room, add/delete staff and dashboard statistics.



Dashboard is part of the regression suite

Retirement Planning

Retirement means replacing, rewriting or migrating the system when maintenance is no longer practical.

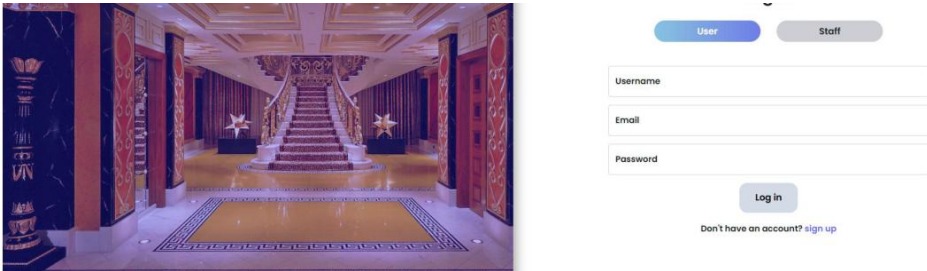
Retirement trigger	Project example	Recommended action
Drastic design change	Hotel needs multi-branch cloud system with advanced roles and payment integration.	Rewrite using more scalable architecture/framework.
New environment	System must run on a different platform or modern hosting stack incompatible with current code.	Migrate and refactor code.
Missing/inaccurate documentation	Future developers cannot understand current system behavior.	Update documentation or rewrite core modules.
Cheaper to rewrite than modify	Security and scalability changes become too large for the procedural PHP structure.	Start a new version using a modern framework and preserve database migration plan.

Appendix A and B: Installation Guide and Test Accounts

The report provides local setup steps and test credentials.

Step	Action
1	Install XAMPP and start Apache and MySQL.
2	Copy project folder to C:\xampp\htdocs\Hotel-Management-System.
3	Open http://localhost/phpmyadmin.
4	Create a database named bluebirdhotel.
5	Import bluebirdhotel.sql into the database.
6	Verify config.php uses localhost, root, empty password and bluebirdhotel.
7	Open http://localhost/Hotel-Management-System/index.php.
8	Log in using test accounts and execute test cases.

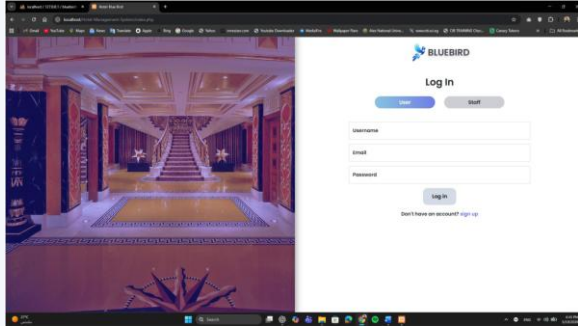
Role	Email	Password
Admin/Staff	admin@gmail.com	1234
User	test@gmail.com	123
User	mohamed@gmail.com	mohamed@gmail.com



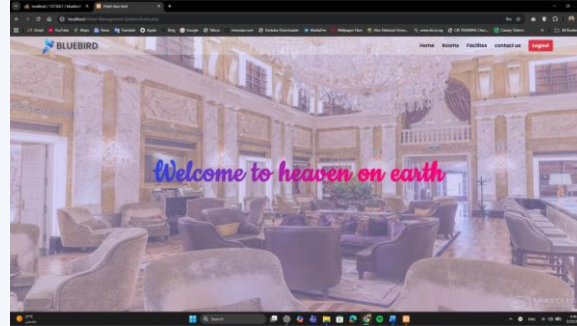
Login screen from the report screenshots

User Interface Screenshots

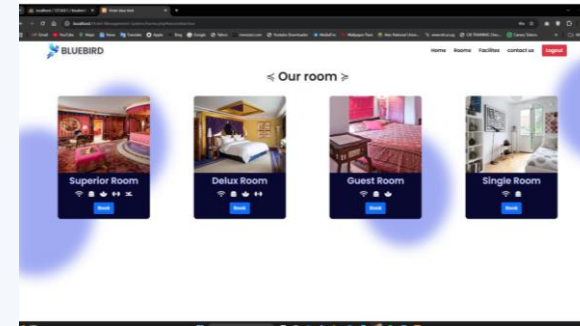
Report screenshots show the user login, homepage, rooms, reservation and facilities screens.



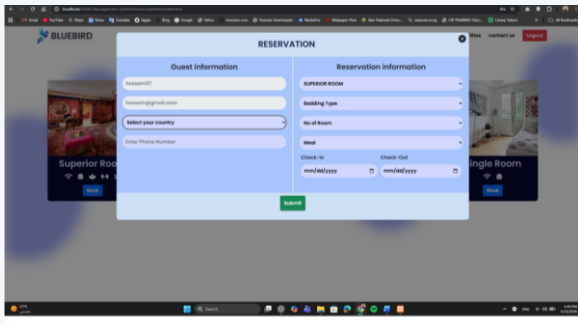
Login



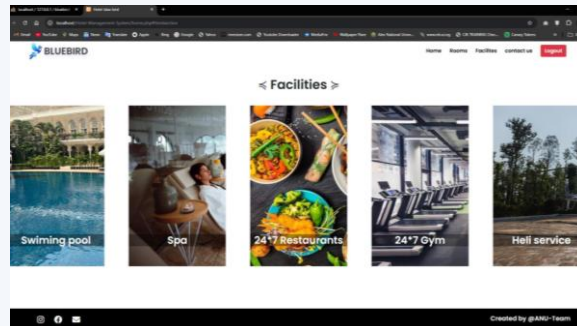
Homepage



Rooms



Reservation



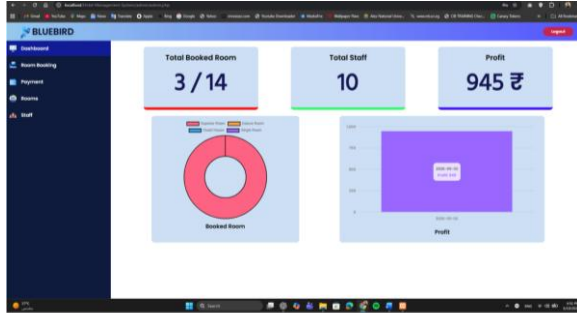
Facilities

User flow

Login → homepage → rooms → reservation form → facilities. The screenshots demonstrate the public website workflow and visual design.

Admin Interface Screenshots

Admin screenshots cover dashboard, booking management, payments and invoice printing.



Dashboard

The bookings page displays a table of reservations with columns for Name, Email, Country, Phone, Type of Room, Type of Bed, Room Number, Rate, Check-In, Check-Out, and Status. The table lists several bookings, including one for 'Mohammed' and another for 'hassan07'.

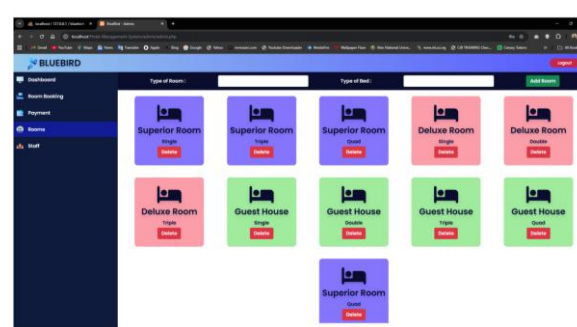
Bookings

The payments page displays a table of transactions with columns for Name, Room Type, Bed Type, Check-In, Check-Out, No. of Days, No. of Rooms, Room Rate, Bed Rate, Rate, and Action. The table lists several transactions, including one for 'Mohammed' and another for 'hassan07'.

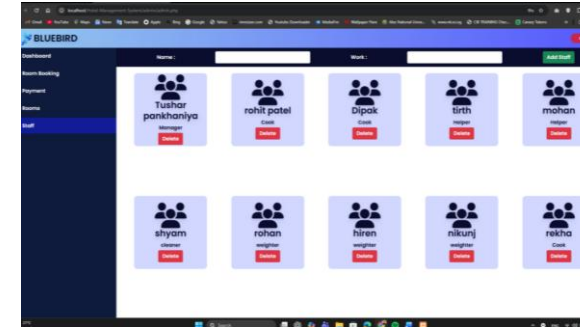
Payments

The invoice page displays a detailed bill for a guest named 'Muhammad'. It includes a table with columns for Item, No. of Days, Rate, Quantity, and Price. The table lists several items, including 'Superior Room', 'Deluxe Room', and 'Guest House'. The total amount is 945.00 ₹.

Invoice



Rooms



Staff

Admin modules

Dashboard, booking, payment, room and staff pages implement the administration workflow.

Final Deliverables Checklist

The report marks all required deliverables as complete.

Deliverable	Status
Working Software Application	✓
Project Report	✓
Use Case Diagram	✓
Sequence Diagram	✓
Activity Diagram	✓
Class Diagram	✓
State Machine Diagram	✓
Test Cases	✓
Screenshots	✓



Thank You!